

# Mini-ChatGPT: Повна теорія та архітектура (Етап 1)

Документ створено для детального опису системи генерації тексту на основі мовних моделей.

---

## Зміст

|          |                                             |          |
|----------|---------------------------------------------|----------|
| <b>1</b> | <b>Вступ</b>                                | <b>3</b> |
| 1.1      | Що таке Mini-ChatGPT? . . . . .             | 3        |
| 1.2      | Етап 1: Що реалізовано . . . . .            | 3        |
| <b>2</b> | <b>Архітектура проекту</b>                  | <b>3</b> |
| 2.1      | Структура рішення . . . . .                 | 3        |
| 2.2      | Граф залежностей . . . . .                  | 4        |
| <b>3</b> | <b>Контракти та інтерфейси</b>              | <b>4</b> |
| 3.1      | ITokenizer . . . . .                        | 4        |
| 3.2      | ILanguageModel . . . . .                    | 5        |
| 3.3      | ITextGenerator . . . . .                    | 5        |
| 3.4      | Checkpoint . . . . .                        | 5        |
| <b>4</b> | <b>Корпус тексту</b>                        | <b>6</b> |
| 4.1      | Що таке корпус? . . . . .                   | 6        |
| 4.2      | CorpusLoader . . . . .                      | 6        |
| 4.3      | CorpusLoadOptions . . . . .                 | 6        |
| 4.4      | CorpusSplitter . . . . .                    | 6        |
| 4.5      | Приклад . . . . .                           | 6        |
| <b>5</b> | <b>Токенізація</b>                          | <b>7</b> |
| 5.1      | WordTokenizer (словний рівень) . . . . .    | 7        |
| 5.2      | CharTokenizer (символьний рівень) . . . . . | 7        |

|           |                                         |           |
|-----------|-----------------------------------------|-----------|
| 5.3       | Порівняння . . . . .                    | 8         |
| <b>6</b>  | <b>Моделі мовлення</b>                  | <b>8</b>  |
| 6.1       | Bigram (біграмна модель) . . . . .      | 8         |
| 6.2       | Trigram (триграмна модель) . . . . .    | 9         |
| 6.3       | TinyNN (нейронна мережа) . . . . .      | 9         |
| 6.4       | TinyTransformer (трансформер) . . . . . | 10        |
| <b>7</b>  | <b>Математичні операції</b>             | <b>12</b> |
| 7.1       | Softmax . . . . .                       | 12        |
| 7.2       | Cross-Entropy Loss . . . . .            | 12        |
| 7.3       | ArgMax . . . . .                        | 12        |
| 7.4       | SampleFromProbs . . . . .               | 12        |
| <b>8</b>  | <b>Семплінг</b>                         | <b>12</b> |
| 8.1       | Задача . . . . .                        | 12        |
| 8.2       | Temperature (температура) . . . . .     | 13        |
| 8.3       | Top-K . . . . .                         | 13        |
| 8.4       | Алгоритм Sampler . . . . .              | 13        |
| <b>9</b>  | <b>Чекпоінт</b>                         | <b>13</b> |
| 9.1       | Формат . . . . .                        | 13        |
| 9.2       | modelPayload по моделях . . . . .       | 14        |
| 9.3       | ContractFingerprintChain . . . . .      | 14        |
| <b>10</b> | <b>Тренування</b>                       | <b>14</b> |
| 10.1      | CLI Trainer . . . . .                   | 14        |
| 10.2      | Потік тренування . . . . .              | 14        |
| 10.3      | TrainTinyNN (детально) . . . . .        | 15        |
| <b>11</b> | <b>Чат та генерація тексту</b>          | <b>15</b> |
| 11.1      | CLI Chat . . . . .                      | 15        |
| 11.2      | ModelTextGenerator.Generate . . . . .   | 15        |
| 11.3      | Repetition Penalty . . . . .            | 16        |
| 11.4      | Що таке REPL? . . . . .                 | 16        |
| 11.5      | Команди REPL . . . . .                  | 16        |

|                                         |           |
|-----------------------------------------|-----------|
| <b>12 Повний потік даних</b>            | <b>16</b> |
| 12.1 Тренування . . . . .               | 16        |
| 12.2 Інференс (чат) . . . . .           | 17        |
| <b>Додаток А: Приклад повного циклу</b> | <b>17</b> |
| <b>Додаток В: Глосарій</b>              | <b>18</b> |

# 1 Вступ

## 1.1 Що таке Mini-ChatGPT?

**Mini-ChatGPT** — це навчальний проєкт, що демонструє побудову системи генерації тексту, подібної до ChatGPT, але в мінімальному масштабі. Система складається з:

- **Мовних моделей** — алгоритмів, що передбачають наступне слово (токен) за контекстом
- **Токенізаторів** — перетворення тексту на числові ідентифікатори
- **Тренера** — навчання моделей на корпусі тексту
- **Чату** — інтерактивної генерації відповідей на промпти

## 1.2 Етап 1: Що реалізовано

| Компонент                     | Опис                                                         |
|-------------------------------|--------------------------------------------------------------|
| <b>Bigram</b>                 | Модель, що враховує лише попереднє слово                     |
| <b>Trigram</b>                | Модель з двома попередніми словами                           |
| <b>TinyNN</b>                 | Нейронна мережа: ембеддинги + лінійний шар, з тренуванням    |
| <b>TinyTransformer</b>        | Міні-трансформер: self-attention + FFN (поки без тренування) |
| <b>Word/Char токенізатори</b> | Рівень слів або символів                                     |
| <b>Семплінг</b>               | Temperature + Top-K для вибору наступного токена             |

# 2 Архітектура проєкту

## 2.1 Структура рішення

```
src/
|-- Contracts/           # Інтерфейси, Checkpoint
|-- Lib.Corporus/       # Завантаження корпусу
```

```

|-- Lib.Tokenization/      # Токенізація
|-- Lib.MathCore/          # Softmax, loss, семплінг
|-- Lib.Sampling/          # Temperature, Top-K
|-- Lib.Models.NGram/      # Bigram, Trigram
|-- Lib.Models.TinyNN/     # Нейронна мережа
|-- Lib.Models.TinyTransformer/ # Трансформер
|-- Lib.Batching/          # Батчі для тренування
|-- Lib.Training/          # Цикл тренування
|-- Lib.ChatConsole/       # REPL чату
|-- Trainer/               # CLI тренування
'-- Chat/                  # CLI чату

```

## 2.2 Граф залежностей

Contracts (корінь)

```

|-- Lib.Corpus
|-- Lib.Tokenization
|-- Lib.MathCore
|   |-- Lib.Models.TinyNN
|   '--- Lib.Sampling
|-- Lib.Models.NGram
|-- Lib.Models.TinyNN
|-- Lib.Models.TinyTransformer
|-- Lib.Training
'-- Lib.ChatConsole

```

Trainer → Corpus, Tokenization, NGram, TinyNN, TinyTransformer, Training  
 Chat → ChatConsole, NGram, TinyNN, TinyTransformer,  
 Sampling, Tokenization

## 3 Контракти та інтерфейси

### 3.1 ITokenizer

```

public interface ITokenizer
{
    int VocabSize { get; }
    int[] Encode(string text);           // текст → масив ID токенів
    string Decode(ReadOnlySpan<int> tokens); // ID → текст
    object GetPayloadForCheckpoint();    // для збереження в чекпоінт
}

```

**Призначення:** Перетворює текст на числові ідентифікатори та назад. Кожен токен (слово або символ) має унікальний ID від 0 до VocabSize-1.

**Приклад:**

Текст: "Привіт світ"  
Encode → [5, 12] (якщо "Привіт"=5, "світ"=12)  
Decode([5, 12]) → "Привіт світ"

## 3.2 ILanguageModel

```
public interface ILanguageModel
{
    string ModelKind { get; }    // "ngram", "trigram",
                                // "tinynn", "tinytransformer"
    int VocabSize { get; }
    float[] NextTokenScores(ReadOnlySpan<int> context);
    // логіти для кожного токена
    object GetPayloadForCheckpoint();
}
```

**Призначення:** За контекстом (масив ID токенів) повертає “оцінки” (логіти) для кожного можливого наступного токена. Вищий логіт = більша ймовірність.

**Приклад:**

Контекст: [5] (одне слово "Привіт")  
NextTokenScores([5]) → [0.01, 0.02, ...,  
0.15, ..., 0.03] (VocabSize елементів)

## 3.3 ITextGenerator

```
public interface ITextGenerator
{
    string Generate(string prompt, int maxTokens,
        float temperature, int topK, int? seed = null);
}
```

**Призначення:** Генерує продовження тексту за промптом. Використовує модель, токенізатор і семплер.

## 3.4 Checkpoint

```
public record Checkpoint(
    string ModelKind,                // "ngram", "trigram",
                                    // "tinynn", "tinytransformer"
    string TokenizerKind,           // "word" або "char"
    object TokenizerPayload,        // словник/словник слів
    object ModelPayload,           // ваги/ймовірності моделі
    int Seed,
    string ContractFingerprintChain
);
```

**Призначення:** Зберігає все необхідне для відтворення моделі та токенизатора після перезапуску.

## 4 Корпус тексту

### 4.1 Що таке корпус?

**Корпус** — це набір текстів, на яких тренується модель. У Mini-ChatGPT корпус завантажується з файлу та розбивається на тренувальну та валідаційну частини.

### 4.2 CorpusLoader

Кроки завантаження:

1. **Читання файлу** — `File.ReadAllText(path)`
2. **Нормалізація** — опційно приведення до нижнього регістру
3. **Розділення** — 90% тренування, 10% валідації (за замовчуванням)

### 4.3 CorpusLoadOptions

`CorpusLoadOptions(lowercase: bool, validationFraction: double)`

- `lowercase = true` — усі слова в нижній регістр (зменшує розмір словника)
- `validationFraction = 0.1` — 10% тексту для валідації

### 4.4 CorpusSplitter

Розділяє текст за довжиною:

Текст: "АБВГДЕЄЖЗИ..." (1000 символів)  
`Split(0.1)` → `TrainText`: перші 900 символів  
                  `ValText`: останні 100 символів

### 4.5 Приклад

Файл `data/showcase.txt`:

Ой у лузі червона калина похилилася.  
Чогось наша славна Україна журиться.  
Привіт! Добрий день! Як справи?

Після завантаження:

```
var corpus = CorpusLoader.Load("data/showcase.txt");  
// corpus.TrainText = "Ой у лузі червона калина..." (90%)  
// corpus.ValText = "..." (10%)
```

## 5 Токенізація

### 5.1 WordTokenizer (словний рівень)

**Принцип:** Текст розбивається на слова за пробілами. Кожне унікальне слово отримує ID.

**Побудова словника (BuildFromText):**

1. Розбити текст: `Regex.Split(text, @"\s+")` → масив слів
2. Для кожного слова:
  - Якщо слово нове → присвоїти наступний ID
  - Додати варіанти: без пунктуації (., !, ?), нижній регістр — з тим самим ID

**Приклад:**

Текст: "Привіт привіт! Як справи?"

Слова: ["Привіт", "привіт!", "Як", "справи?"]

Словник:

"привіт" → 0 (включає "Привіт", "привіт!", "привіт")

"як" → 1

"справи" → 2 (включає "справи?")

**Encode:**

"Привіт як" → [0, 1]

**Decode:**

[0, 1] → "привіт як"

### 5.2 CharTokenizer (символьний рівень)

**Принцип:** Кожен символ — окремий токен. Словник будується з усіх унікальних символів у тексті.

**Приклад:**

Текст: "Привіт"

Словник: {'П'→0, 'р'→1, 'и'→2, 'в'→3, 'і'→4, 'т'→5, ...}

Encode("Привіт") → [0, 1, 2, 3, 4, 5]

Decode([0,1,2,3,4,5]) → "Привіт"

## 5.3 Порівняння

| Аспект                | Word                | Char                         |
|-----------------------|---------------------|------------------------------|
| Розмір словника       | Кілька сотень–тисяч | ~50–200                      |
| Довжина послідовності | Коротша             | Довша                        |
| Генерація             | Цілі слова          | Може “вигадувати” нові слова |
| Швидкість             | Швидше              | Повільніше                   |

# 6 Моделі мовлення

## 6.1 Bigram (біграмна модель)

**Ідея:** Ймовірність наступного слова залежить лише від попереднього.

**Математично:**

$$P(w_n | w_{n-1}) = \frac{\text{count}(w_{n-1}, w_n)}{\sum_w \text{count}(w_{n-1}, w)}$$

**Структура даних:** Матриця `_probs[prev][next]` розміру `VocabSize × VocabSize`.

**Навчання:**

1. Підрахувати пари (`prev`, `next`) у корпусі
2. Для кожного `prev`: нормалізувати рядок (сума = 1)

**Приклад:**

Корпус: "привіт світ привіт дім"

Пари: (привіт, світ), (світ, привіт), (привіт, дім)

```
count[привіт][світ] = 1
count[привіт][дім] = 1
count[світ][привіт] = 1
```

$P(\text{світ}|\text{привіт}) = 1/2$

$P(\text{дім}|\text{привіт}) = 1/2$

$P(\text{привіт}|\text{світ}) = 1$

**NextTokenScores:**

Контекст: [привіт]

Повертаємо `_probs[привіт]` - рядок ймовірностей для всіх слів



## 6.2 Trigram (триграмна модель)

**Ідея:** Ймовірність залежить від двох попередніх слів.

**Математично:**

$$P(w_n \mid w_{n-2}, w_{n-1}) = \frac{\text{count}(w_{n-2}, w_{n-1}, w_n)}{\sum_w \text{count}(w_{n-2}, w_{n-1}, w)}$$

**Структура:**

- `_bigramProbs[prev][next]` — fallback, коли триграма не знайдена
- `_trigramProbs[(p2, p1)]` — словник: пара (передостаннє, останнє)  $\rightarrow$  масив ймовірностей

**Логіка NextTokenScores:**

1. Якщо контекст  $\geq 2$ : шукати триграму (p2, p1)
2. Якщо знайдено — повернути її ймовірності
3. Інакше — fallback на біграму (останнє слово)

**Приклад:**

Контекст: [у, лузі]

Триграма (у, лузі)  $\rightarrow$  P(червона)=0.7, P(зелена)=0.3, ...

Контекст: [лузі]

Лише одне слово  $\rightarrow$  біграма P(next|лузі)

## 6.3 TinyNN (нейронна мережа)

**Архітектура:** Embedding  $\rightarrow$  усереднення  $\rightarrow$  Linear  $\rightarrow$  логіти.

**Компоненти**

**EmbeddingLayer:**

- Таблиця ембеддингів: `Embeddings[token_id]` — вектор довжини `EmbeddingSize` (32)
- Контекст кодується як **середнє** ембеддингів усіх токенів контексту

**LinearHead:**

- Лінійне перетворення: `logits = hidden * OutputWeights + OutputBias`
- Розмірність: `hidden` (32)  $\rightarrow$  `vocab` (`VocabSize`)

## Конфігурація

`TinyNNConfig(VocabSize, EmbeddingSize=32, ContextSize=8)`

- **ContextSize=8** — використовуємо лише останні 8 токенів (якщо контекст довший)

## Прямий прохід (Forward)

Контекст: [5, 12, 3] (3 слова)

1. Embedding:  $e_5, e_{12}, e_3$  - вектори 32-вимірні
2. Усереднення:  $hidden = (e_5 + e_{12} + e_3) / 3$
3. Linear:  $logits = hidden \times W + b \rightarrow VocabSize$  чисел

## Зворотний прохід (Backward) — тренування

**Крок 1:** Forward  $\rightarrow logits, probs = \text{softmax}(logits)$

**Крок 2:** Loss =  $-\log(probs[target])$  (cross-entropy для одного класу)

**Крок 3:** Градієнт логітів:

$dLogits[i] = probs[i] - (i == target ? 1 : 0)$

**Крок 4:** `LinearHead.Backward` — оновлює `OutputWeights` `OutputBias`; повертає `dHidden`

**Крок 5:** `EmbeddingLayer.Backward` — розподіляє `dHidden` на ембеддинги контекстних токенів (з коефіцієнтом `lr/count`)

**SGD оновлення:**

`weight -= lr * gradient`

## 6.4 TinyTransformer (трансформер)

**Архітектура:** Один блок: Self-Attention  $\rightarrow$  Feed-Forward  $\rightarrow$  вихідна проекція.

### SelfAttentionLayer (каузальна self-attention)

**Кроки:**

1. **Embedding контексту:** кожен токен  $\rightarrow$  вектор  $d = 16$

- $x[i] = \text{TokenEmbeddings}[\text{context}[i]]$

2. Q, K, V:

- $Q = x \times W_q$  (запит)
- $K = x \times W_k$  (ключ)
- $V = x \times W_v$  (значення)

### 3. Скалярний добуток з масштабуванням:

- $\text{scores}[i][j] = (Q[i] \cdot K[j]) / \sqrt{d}$  якщо  $j \leq i$ , інакше  $-\infty$  (каузальна маска)

4. **Softmax по рядках:**  $\text{attn}[i] = \text{softmax}(\text{scores}[i])$

5. **Зважена сума V:**  $\text{out}[i] = \sum_j \text{attn}[i][j] * V[j]$

6. **Вихідна проекція:**  $\text{proj} = \text{out} \times W_o$

7. **Повертаємо останню позицію:**  $\text{proj}[n-1]$  — вектор для останнього токена

**Каузальна маска:** Позиція  $i$  може “бачити” лише позиції  $0..i$ , не майбутні.

### FeedForwardLayer

$\text{hidden } (d) \rightarrow \text{Linear } (d \rightarrow 4d) \rightarrow \text{ReLU}$   
 $\rightarrow \text{Linear } (4d \rightarrow d) \rightarrow \text{Linear } (d \rightarrow \text{vocab}) \rightarrow \text{logits}$

- **Ffn1:**  $d \times 4d$ , bias Ffn1Bias
- **Ffn2:**  $4d \times d$ , bias Ffn2Bias
- **OutputW, OutputBias:**  $d \times \text{vocab}$

**ReLU:**  $\max(0, x)$

### Конфігурація

`TinyTransformerConfig(VocabSize, EmbeddingSize=16,  
HeadCount=1, ContextSize=8)`

### ContextSize

Як і в TinyNN, якщо контекст довший за 8 токенів, беремо лише останні 8.

### Стан тренування

**TinyTransformer:** зараз лише інференс (без тренування). Ваги ініціалізуються випадково, тому модель не навчена.

## 7 Математичні операції

### 7.1 Softmax

Перетворює логіти на ймовірності (сума = 1):

$$\text{softmax}(z_i) = \frac{e^{z_i}}{\sum_j e^{z_j}}$$

**Числова стабільність:** віднімаємо  $\max(z)$  перед `exp`, щоб уникнути overflow.

```
// Псевдокод
max = max(logits)
exp[i] = exp(logits[i] - max)
sum = sum(exp)
probs[i] = exp[i] / sum
```

### 7.2 Cross-Entropy Loss

Для одного правильного класу (target):

$$L = -\log(p_{\text{target}})$$

Де  $p$  — ймовірність з softmax.

### 7.3 ArgMax

Повертає індекс максимального елемента — для “жадібного” вибору наступного токена.

### 7.4 SampleFromProbs

Вибір індексу з дискретного розподілу за ймовірностями (random sampling).

## 8 Семплінг

### 8.1 Задача

Модель повертає логіти. Потрібно вибрати один токен для генерації. Можливі підходи:

1. **ArgMax** — завжди найбільш ймовірний (передбачувано, повторювано)
2. **Random** — пропорційно ймовірностям (різноманітно, але нестабільно)

**Temperature** + **Top-K** — компроміс між передбачуваністю та різноманіттям.

## 8.2 Temperature (температура)

Масштабування логітів перед softmax:

$$\tilde{z}_i = \frac{z_i}{T}$$

- $T < 1$  — різкіша розподіл, менше випадковості
- $T = 1$  — без змін
- $T > 1$  — м'якший розподіл, більше випадковості

**Реалізація:** `logits[i] = log(probs[i]) / temperature` (після softmax знову логіти)

## 8.3 Top-K

Залишаємо лише  $K$  найбільших логітів, решту обнуляємо. Потім softmax за нормалізацією.

**Приклад:** При  $K=5$  і 100 токенах залишаємо 5 найкращих, інші ймовірності = 0.

## 8.4 Алгоритм Sampler

1. `probs` = вхідні ймовірності
2. `tempered` = `log(probs) / temperature`
3. `indices` = `TopKSelector(tempered, topK)` - індекси  $K$  найбільших
4. `topKProbs` = `exp(tempered[indices])`
5. `topKProbs` = `normalize(topKProbs)`
6. `idx` = `SampleFromProbs(topKProbs)` - випадковий вибір
7. `return indices[idx]`

## 9 Чекпоінт

### 9.1 Формат

Чекпоінт зберігається у JSON і містить:

```
{
  "modelKind": "trigram",
  "tokenizerKind": "word",
  "tokenizerPayload": { "words": ["привіт", "світ", ...] },
  "modelPayload": { "bigramProbs": [...], "trigramProbs": [...] },
  "seed": 42,
  "contractFingerprintChain": "Lib.Corporus:...|Lib.Tokenization:...|..."
}
```

## 9.2 modelPayload по моделях

| Модель          | modelPayload                                                          |
|-----------------|-----------------------------------------------------------------------|
| bigram          | Матриця ймовірностей [vocab] [vocab]                                  |
| trigram         | bigramProbs + trigramProbs (список пар та ймовірностей)               |
| tinynn          | config + embeddings, outputWeights, outputBias                        |
| tinytransformer | config + tokenEmbeddings, Wq, Wk, Wv, Wo, Ffn1, Ffn2, OutputW, biases |

## 9.3 ContractFingerprintChain

Ланцюжок відбитків версій контрактів (Corpus, Tokenizer, Model) для перевірки сумісності при завантаженні.

# 10 Тренування

## 10.1 CLI Trainer

```
dotnet run --project src/Trainer -- train \
  --data data/showcase.txt \
  --model bigram|trigram|tinynn|tinytransformer \
  --tokenizer word|char \
  --epochs 30 \
  --lr 0.05 \
  --out checkpoint.json \
  --seed 42
```

## 10.2 Потік тренування

1. Завантаження корпусу — `CorpusLoader.Load(path)`
2. Побудова токенизатора — `WordTokenizer.BuildFromText`  
або `CharTokenizer.BuildFromText`
3. Кодування — `tokenizer.Encode(corpus.TrainText) → int[]`
4. Тренування моделі:
  - **bigram/trigram:** `Train(tokens)` — підрахунок n-грам
  - **tinynn:** для кожного (context, target) виклик `TrainStep(context, target, lr)`
  - **tinytransformer:** лише створення моделі (без тренування)
5. Збереження чекпоінту — `JsonCheckpointIO.Save`

## 10.3 TrainTinyNN (детально)

Для кожного епохи:

```
Для кожного i від 0 до len(tokens)-1:
    context = tokens[0..i+1]
    target = tokens[i+1]
    loss = model.TrainStep(context, target, lr)
    totalLoss += loss
Вивести середній loss
```

**Приклад:** Корпус “привіт світ привіт дім” → пари (привіт, світ), (привіт світ, привіт), (привіт світ привіт, дім), ...

## 11 Чат та генерація тексту

### 11.1 CLI Chat

```
dotnet run --project src/Chat -- chat \
--checkpoint checkpoint.json \
--temp 0.3 \
--topk 5 \
--seed 42
```

### 11.2 ModelTextGenerator.Generate

**Вхід:** prompt, maxTokens, temperature, topK, seed

**Алгоритм:**

1. context = tokenizer.Encode(prompt)
2. result = [context] (копія)
3. Цикл до maxTokens:
  - scores = model.NextTokenScores(result)
  - probs = softmax(scores)
  - **Repetition penalty:** для останніх 8 токенів зменшити їхні ймовірності в 1.5 рази
  - Нормалізувати probs
  - next = Sampler.Sample(probs, temp, topK)
  - result.Add(next)
  - **Умова зупинки:** якщо newCount >= 5 і останній токен — . або ! або ? → break
4. Повернути tokenizer.Decode(result[context.Length..])

## 11.3 Repetition Penalty

Щоб уникнути повторень, ймовірності токенів, що вже з'явилися в останніх 8 позиціях, діляться на 1.5.

## 11.4 Що таке REPL?

**REPL** (Read-Eval-Print Loop) — це інтерактивний режим роботи програми, де вона циклічно:

1. **Read** — читає введення користувача;
2. **Eval** — обробляє його;
3. **Print** — виводить результат;
4. **Loop** — знову чекає на введення.

У Mini-ChatGPT чат працює як REPL: ви вводите текст → модель генерує відповідь → результат виводиться → можна вводити наступне повідомлення.

**Команди REPL** — це спеціальні інструкції, що починаються з /. Вони керують середовищем чату, а не передаються в модель як звичайний текст.

## 11.5 Команди REPL

| Команда | Опис                |
|---------|---------------------|
| /reset  | Скинути контекст    |
| /seed N | Встановити seed     |
| /temp T | Змінити температуру |
| /topk K | Змінити top-K       |
| /help   | Допомога            |
| /quit   | Вихід               |

## 12 Повний потік даних

### 12.1 Тренування

```
Файл (data/showcase.txt)
  ↓ CorpusLoader
Corpus(TrainText, ValText)
  ↓ WordTokenizer.BuildFromText
ITokenizer
  ↓ Encode
int[] tokens
  ↓ Model.Train / TrainStep
```



```
ILanguageModel (навчена)
  ↓ GetPayloadForCheckpoint
Checkpoint
  ↓ JsonCheckpointIO.Save
checkpoint.json
```

## 12.2 Інференс (чат)

```
checkpoint.json
  ↓ JsonCheckpointIO.Load
Checkpoint
  ↓ FromPayload (Tokenizer, Model)
ITokenizer, ILanguageModel
  ↓
Користувач: "Привіт"
  ↓ Encode
context = [5]
  ↓ NextTokenScores
scores = [0.01, 0.15, ...]
  ↓ Softmax
probs = [0.02, 0.12, ...]
  ↓ Sampler.Sample(temp, topK)
next = 12
  ↓ Decode
"світ"
  ↓ (повторити в циклі)
"світ прекрасний день"
```

## Додаток А: Приклад повного циклу

### 1. Підготовка даних

Файл data/demo.txt:

Привіт! Як справи? Добрий день.

### 2. Тренування trigram

```
dotnet run --project src/Trainer -- train \
  --data data/demo.txt --model trigram --tokenizer word \
  --epochs 1 --out checkpoint.json
```

Що відбувається:

- Словник: привіт, як, справи, добрий, день

- Триграми: (привіт, як), (як, справи), (справи, добрий), (добрий, день)
- Біграми: fallback для невідомих пар

### 3. Чат

```
dotnet run --project src/Chat -- chat \
  --checkpoint checkpoint.json --temp 0.5 --topk 3
```

Ввід: “Привіт”

Можлива відповідь: “як” (бо (привіт, як) була в корпусі)

## Додаток В: Глосарій

| Термін               | Опис                                                                                      |
|----------------------|-------------------------------------------------------------------------------------------|
| <b>Токен</b>         | Одиниця тексту (слово або символ) з унікальним ID                                         |
| <b>Контекст</b>      | Послідовність токенів, за якою передбачаємо наступний                                     |
| <b>Logit</b>         | Необроблена оцінка перед softmax                                                          |
| <b>Softmax</b>       | Перетворення логітів на ймовірності                                                       |
| <b>Ембеддинг</b>     | Векторне представлення токена                                                             |
| <b>Cross-entropy</b> | Функція втрат для класифікації                                                            |
| <b>SGD</b>           | Stochastic Gradient Descent — оновлення ваг за градієнтом                                 |
| <b>Чекпоінт</b>      | Збережений стан моделі та токенизатора                                                    |
| <b>Промпт</b>        | Вхідний текст від користувача                                                             |
| <b>Temperature</b>   | Параметр “випадковості” генерації                                                         |
| <b>Top-K</b>         | Обмеження вибору K найкращими токенами                                                    |
| <b>REPL</b>          | Read-Eval-Print Loop — інтерактивний цикл: читати введення → обробити → вивести результат |

*Документ створено для етапу 1 проекту Mini-ChatGPT. Версія: 1.0*